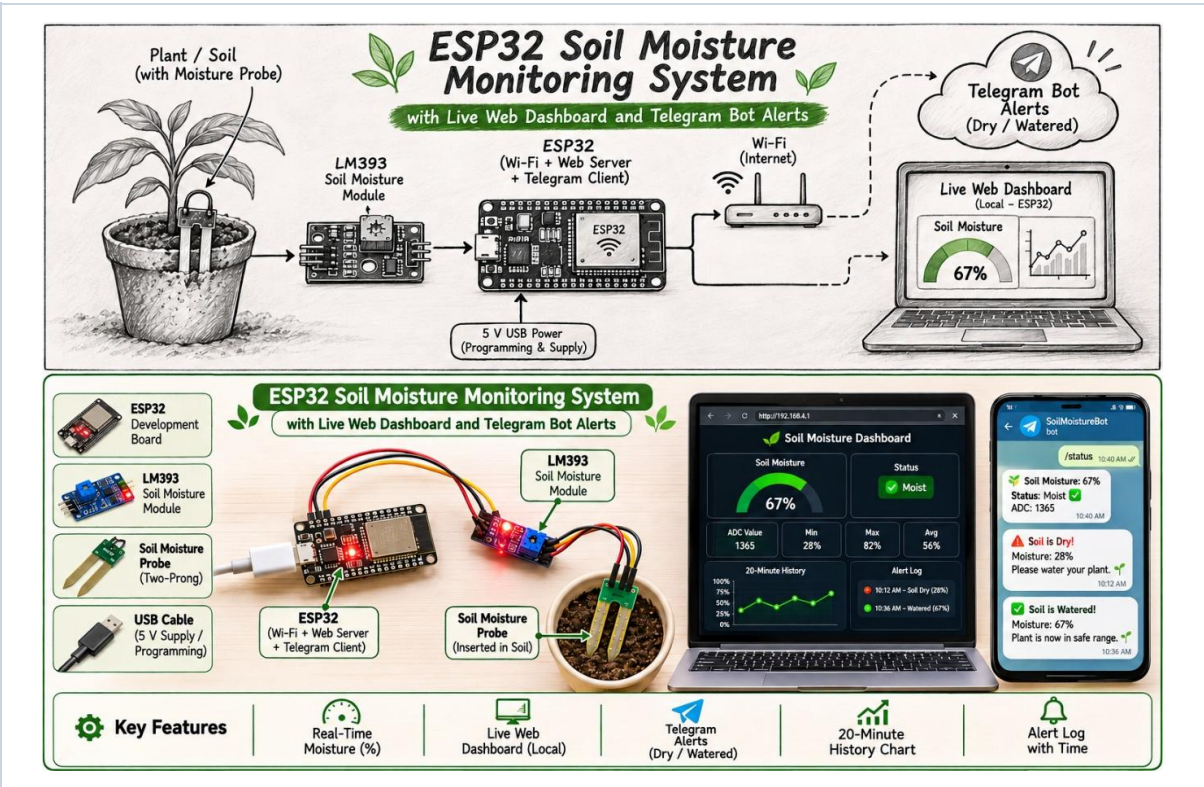
PROJECT DOCUMENTATION

ESP32 Soil Moisture Monitoring System

with Live Web Dashboard and Telegram Bot Alerts

PROJECT BY

PILLA JOSHI



1. Abstract

This project presents an IoT-based soil moisture monitoring system built around the ESP32 microcontroller and a two-prong soil moisture probe with an LM393 comparator module. The ESP32 reads the sensor, converts the raw ADC value into a moisture percentage, and shares that information in two ways at the same time: through a live web dashboard hosted on the ESP32 itself, and through a Telegram bot that sends automatic alerts and answers on-demand commands.

The dashboard works on any phone or computer connected to the same Wi-Fi network and needs no external server or cloud account. It shows a live gauge, device statistics, a 20-minute history chart and an alert log. The Telegram bot sends a warning when the soil becomes dry and a recovery message when it has been watered, so the user is informed even when the dashboard is not open.

2. Introduction

Watering plants at the wrong time, too early or too late, wastes water and harms plant health. Checking the soil by hand is time-consuming and easy to forget. A small embedded system that measures soil moisture continuously and informs the owner only when action is needed solves this problem at very low cost.

The ESP32 is well suited to this task because it combines a 12-bit analog-to-digital converter, built-in Wi-Fi and enough memory to host a complete web page. The soil probe is paired with an LM393 sensor module that provides an analog output, which the ESP32 reads directly, so no extra conversion hardware is needed.

2.1 Problem Statement

> **PROBLEM STATEMENT**
>
> To design a low-cost, self-contained system that measures soil moisture, displays it in real time on a local web dashboard, and notifies the user on Telegram when the soil is dry and when it has recovered.

2.2 Objectives

- Read soil moisture using a soil probe module and the ESP32 ADC.
- Convert the raw reading into an easy-to-understand percentage using a two-point calibration.
- Host a responsive dashboard directly on the ESP32 with live gauge, statistics, history chart and alert log.
- Send automatic dry and recovery alerts to Telegram without flooding the user with repeated messages.
- Allow the user to query the system at any time using Telegram commands.

3. Components and Software Requirements

3.1 Hardware

S.No	Component	Purpose
1	ESP32 development board	Main controller; reads the sensor, runs the web server and the Telegram client over Wi-Fi
2	Soil moisture probe with LM393 module	Two-prong probe whose resistance falls as the soil gets wetter; the module gives an analog output (AO) voltage
3	Jumper wires and breadboard	Connections between the sensor and the ESP32
4	USB cable and 5 V supply	Power and programming
5	Wi-Fi network with internet access	Local dashboard access and Telegram communication

3.2 Software and Libraries

Item	Use
Arduino IDE with ESP32 board package	Programming and uploading the firmware
`WiFi.h`	Connecting the ESP32 to the Wi-Fi network
`WebServer.h`	Serving the dashboard page and the data endpoint on port 80
`WiFiClientSecure.h`	Secure (HTTPS) connection to the Telegram servers
UniversalTelegramBot (Brian Lough)	Sending messages and receiving commands from the Telegram bot
ArduinoJson 6.21.x (Benoit Blanchon)	JSON handling required by the Telegram library
Telegram app, @BotFather, @myidbot	Creating the bot, obtaining the bot token and the user chat ID

4. System Architecture

The sensor output is connected to an analog input of the ESP32. The ESP32 processes the reading and then serves two independent interfaces: an HTTP web server for the dashboard on the local network, and a secure client that talks to the Telegram Bot API over the internet.

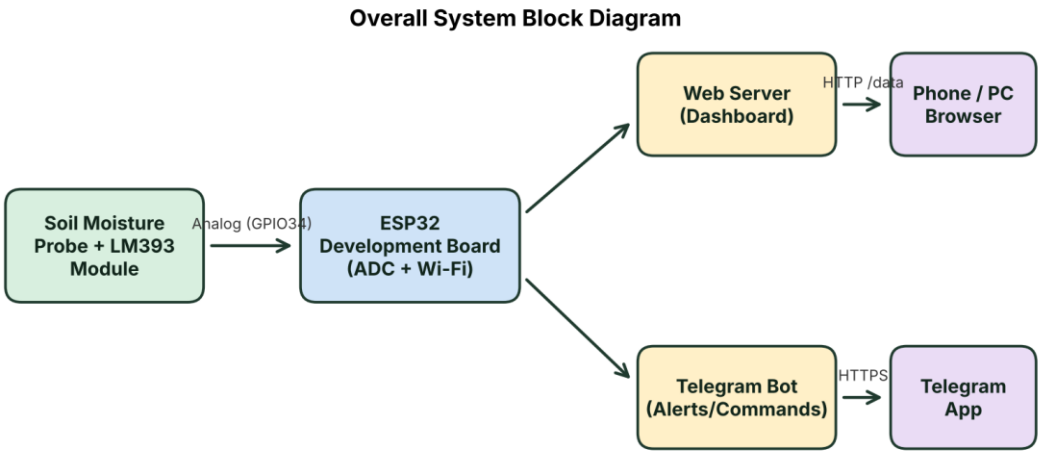


Figure 1: *Overall system block diagram*

4.1 Pin Connections

Sensor pin	ESP32 pin	Description
VCC	3V3	Power supply (3.3 V)
GND	GND	Common ground
AOUT	GPIO34	Analog output to ADC input

> **WHY GPIO34?**
> GPIO34 is an input-only pin on the ADC1 group, which keeps working while Wi-Fi is active. This is important because ADC2 pins of the ESP32 cannot be used reliably while Wi-Fi is on.

4.2 Hardware Prototype

The prototype was assembled on a table using jumper wires. The soil probe connects to the LM393 module, and the module connects to the ESP32 through three wires (power, ground and analog output). The ESP32 is powered through its USB Type-C port. The two red LEDs on the module are its own power and output indicators.

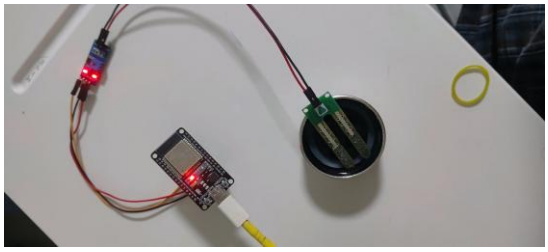


Figure 2: *Probe placed in the test container*

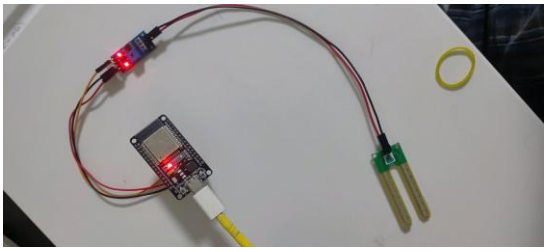


Figure 3: *Probe in open air*

5. Working Principle

The soil probe has two exposed metal prongs. When pushed into soil, the current that flows between the prongs depends on the water content: wet soil conducts better than dry soil. The LM393 module converts this change into an output voltage. With this setup the output voltage falls as moisture increases, which means a lower ADC value corresponds to wetter soil and a higher ADC value corresponds to drier soil.

5.1 Reading the Sensor

The ADC is set to 12-bit resolution (values 0 to 4095) and the input attenuation is set to 11 dB, which lets the pin measure the full 0 to about 3.3 V range of the sensor. To reduce electrical noise, each reading is the average of 20 samples taken 5 ms apart.

5.2 Calibration and Percentage Conversion

The raw value is converted into a percentage by linear interpolation between two calibration points measured for the sensor in use:

Calibration point	Raw ADC value	Moisture
Dry (probe in open air)	3200	0 %
Wet (probe in a glass of water)	1400	100 %

FORMULA

Moisture (%) = (Raw – 3200) × 100 ÷ (1400 – 3200), limited to the range 0 to 100.

Any reading higher than the dry value is therefore shown as 0 %, and any reading lower than the wet value is shown as 100 %. For example, a raw value of 2855 gives about 19 %, 2141 gives about 58 %, and 1969 gives about 68 %, which match the values observed during testing.

5.3 Moisture Condition Levels

Moisture	Condition shown	Meaning
Below 30 %	Dry – needs water	Alert is triggered
30 % to 59 %	Moist (good)	Healthy range
60 % and above	Wet	Sufficient or excess water

6. Software Design

The firmware is a single Arduino sketch. The dashboard web page is stored in the flash memory of the ESP32, so no SD card, external file system or internet-hosted page is needed. The main loop runs three tasks without blocking each other: handling web requests, polling Telegram, and checking the sensor.

6.1 Program Flow

1. **Startup:** Start the serial monitor, configure the ADC, and connect to the Wi-Fi network.
2. **Initialisation:** Load the Telegram security certificate, start the web server and take the first sensor reading so that the dashboard has data immediately.
3. **Notification:** Send an "online" message to Telegram containing the dashboard address (the IP address of the ESP32).
4. **Main loop:** Serve web requests continuously; check Telegram for new messages every 1 second; read the sensor and evaluate alerts every 10 seconds.
5. **Recovery:** If Wi-Fi is lost, the board reconnects automatically.

6.2 Timing Parameters

Parameter	Value	Purpose
Sensor check interval	10 seconds	How often a new reading is taken and stored
Telegram polling interval	1 second	How quickly commands get a reply
Dashboard refresh	5 seconds	How often the web page fetches new data
History buffer	120 readings	About 20 minutes of data kept in RAM
Alert cooldown	30 minutes	Minimum gap between repeated dry alerts
Samples per reading	20 (5 ms apart)	Noise reduction by averaging

6.3 Alert Logic

The alert logic uses two different thresholds, a technique called hysteresis, so that the system does not send messages repeatedly when the moisture value hovers around a single limit.

- **Dry alert:** When moisture drops below 30 %, a dry alert is sent to Telegram and the system remembers that the soil is dry. While the soil stays dry, the alert is repeated only after the 30-minute cooldown has passed.
- **Recovery message:** The system returns to normal only when moisture rises above 40 %. At that moment a recovery message with the new moisture value is sent.
- **Dead band:** Between 30 % and 40 % no new message is sent, which avoids spamming caused by small fluctuations.

6.4 Data Storage

Every reading is stored in a circular buffer that holds the latest 120 moisture percentages and raw values. When the buffer is full, the newest reading replaces the oldest one. The dashboard reads this buffer to draw the history chart, calculate the average, minimum and maximum, and build the alert log. The history is kept in RAM only, so it restarts from zero whenever the board is reset.

6.5 Web Server

Address	Function
`/`	Returns the complete dashboard page (HTML, CSS and JavaScript) stored in flash memory
`/data`	Returns the current data in JSON format: raw value, Wi-Fi signal strength, uptime, alert thresholds, sampling step and the list of stored moisture percentages. Caching is disabled so the browser always receives fresh data

6.6 Dashboard Features

- **Current moisture:** A semicircular gauge with the moisture percentage and a colour-coded condition label (red for dry, green for moist, blue for wet).
- **Device and statistics:** Raw ADC value, Wi-Fi signal strength in dBm, average moisture, minimum and maximum moisture, uptime and number of stored readings.
- **Moisture history:** A line chart of the stored readings with the dry-alert (red dashed) and recovery (green dashed) levels marked on it.
- **Dry warning banner:** A red banner appears at the top of the page whenever the soil is dry.
- **Alert log:** A list of dry and recovered events with the time they occurred.
- **Connection status and theme:** The page shows "Live" when data is arriving and "Device offline" if the ESP32 cannot be reached. It also adapts to light and dark mode and to phone and desktop screen sizes.

6.7 Telegram Bot

The bot accepts commands only from the authorised chat ID. Any other user receives an "Unauthorized user" reply, so strangers cannot read the data.

Command	Reply
`/start`	Welcome message listing the available commands and the dashboard address
`/moisture`	Current moisture percentage, condition (Dry / Moist / Wet) and raw ADC value
`/status`	Moisture details plus Wi-Fi signal strength, uptime in minutes and the dashboard address
Anything else	"Unknown command" message with a hint to send /start

> **AUTOMATIC MESSAGES**
> An online message on every start-up, a dry alert, and a recovery message.

7. Setup Procedure

1. **Create the Telegram bot:** Open Telegram, create a bot with @BotFather and copy the bot token. Get your personal chat ID from @myidbot.
2. **Install libraries:** Install UniversalTelegramBot and ArduinoJson (6.21.x) from the Arduino Library Manager.
3. **Wire the sensor:** Connect VCC to 3V3, GND to GND and AOUT to GPIO34.
4. **Configure the sketch:** Enter the Wi-Fi name and password, the bot token and the chat ID in the user-settings section of the sketch.
5. **Calibrate:** Note the raw value in open air and in water, and enter them as the dry and wet calibration values.
6. **Upload and run:** Upload to the ESP32, open the Serial Monitor at 115200 baud to see the IP address, and open that address in a browser on the same Wi-Fi network.

8. Results and Testing

The system was tested by changing the condition of the sensor and observing the Telegram messages and the dashboard. The following screenshots were captured during testing.

8.1 Telegram Bot Output

The Telegram chat shows the start-up message with the dashboard address, dry alerts, recovery messages, and the replies to the /moisture and /status commands.



Figure 4: *Telegram bot alerts and command replies*

8.2 Dashboard – Moist Condition

With the sensor in moist soil, the dashboard shows 58 % with the label "Moist (good)", a raw value of 2141 and a Wi-Fi signal of −56 dBm. The chart shows the rise from 0 % to about 63 % after the sensor was placed in moisture.

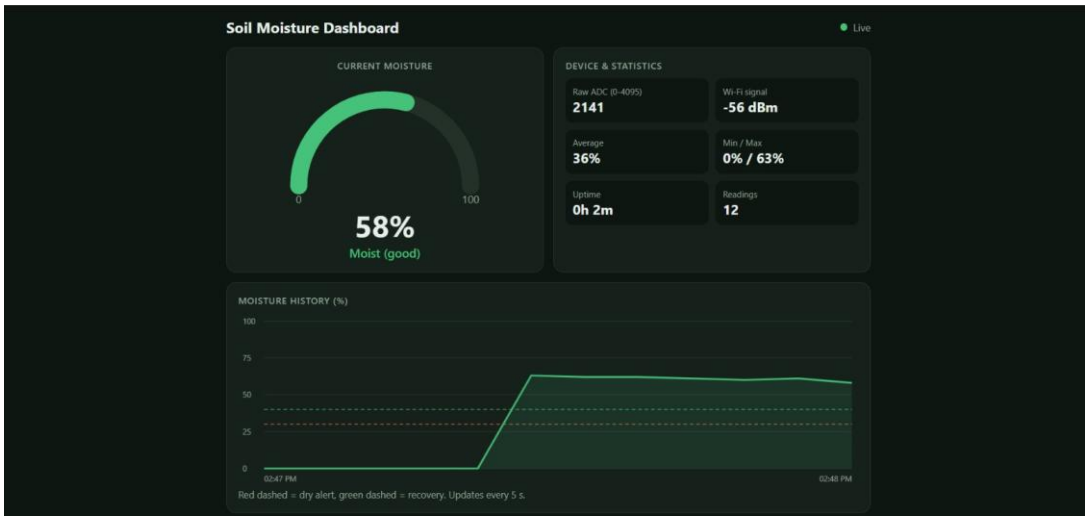


Figure 5: *Dashboard showing moist soil (58 %)*

8.3 Dashboard – Dry Condition and Alert

When the sensor was taken out of the moisture, the reading went to the dry end of the range (raw value 4095, moisture 0 %). The gauge turns red, the label changes to "Dry – needs water" and the red warning banner appears at the top of the page.

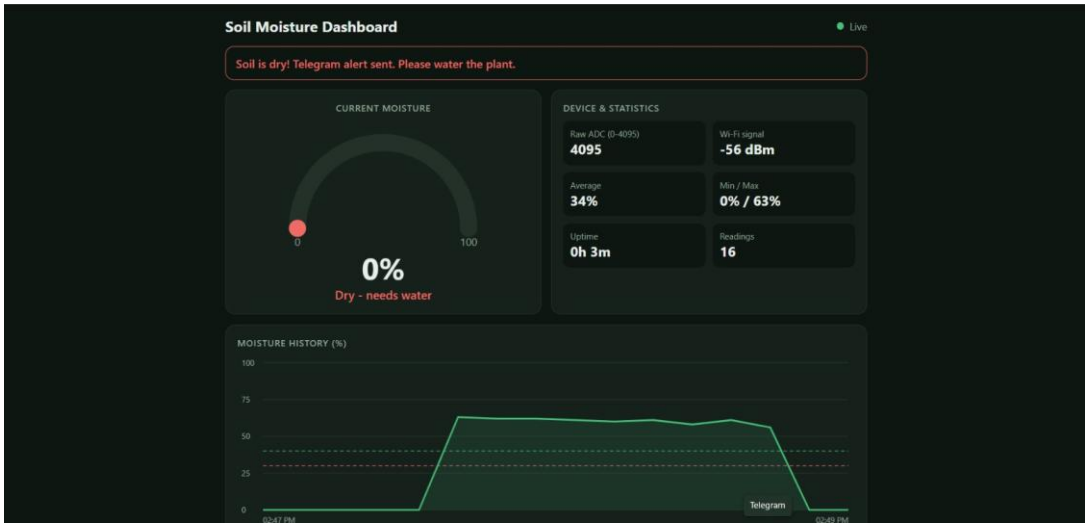


Figure 6: *Dashboard showing dry condition with alert banner*

8.4 Dashboard – Wet Condition

After the sensor was wetted again, the moisture rose to 68 % (raw 1969) and the gauge turned blue with the label "Wet". The history chart clearly shows the full dry, wet, dry and wet cycle.

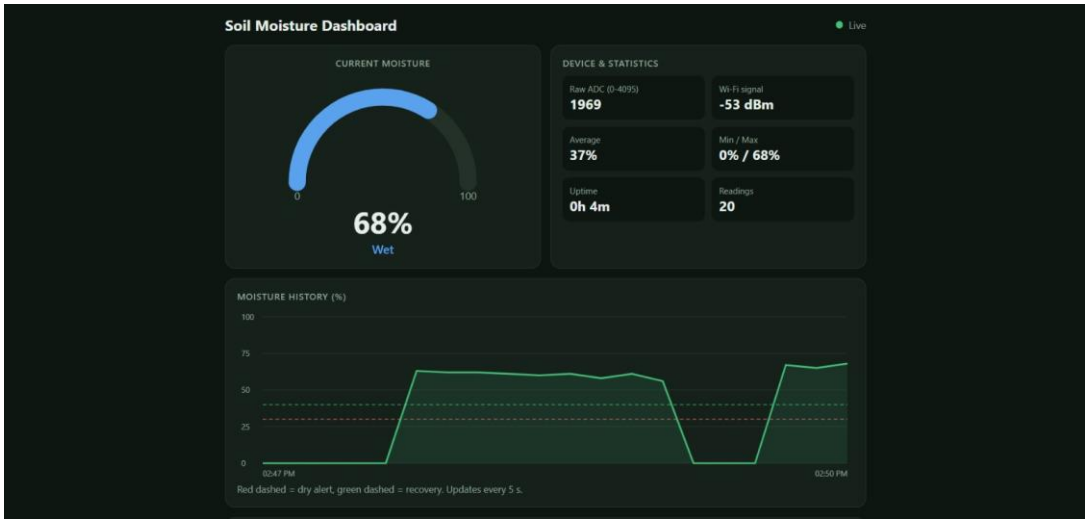


Figure 7: *Dashboard showing wet condition (68 %) and history chart*

8.5 Summary of Observed Readings

Situation	Raw ADC	Moisture	Condition
Sensor dry / out of moisture	4095	0 %	Dry – alert sent
Dry soil (Telegram test)	2855	19 %	Dry
Moist soil	2141	58 %	Moist (good)
Wet soil	1969	68 %	Wet – recovery message

8.6 Observations

- The Telegram dry alert and the dashboard banner appeared together, confirming that both interfaces use the same sensor data.
- Recovery messages (63 % and 67 %) were sent only after moisture crossed the 40 % recovery level, confirming that the two-threshold logic works.
- The /moisture and /status commands returned values that matched the dashboard at the same time (19 %, raw 2855, Wi-Fi −53 dBm).
- Repeated "online" messages in the chat correspond to times the board was restarted during testing.
- The dashboard history chart updated every few seconds and showed the Live status throughout testing.

9. Advantages and Limitations

9.1 Advantages

- Low cost and built from easily available parts.
- No cloud server or subscription needed for the dashboard; it runs on the ESP32 itself.
- Real-time alerts reach the user anywhere through Telegram.
- Simple, low-cost sensing with an analog output that the ESP32 reads directly.
- Alerts use hysteresis and a cooldown to avoid message spam.

9.2 Limitations

- The dashboard is reachable only on the same Wi-Fi network unless port forwarding or a VPN is set up.
- History is stored in RAM and is lost when the board restarts or loses power.
- Resistive probes have exposed metal prongs that can corrode over time in wet soil, so readings may drift and the probe may need replacing.
- The calibration values depend on the individual sensor and soil type and must be re-measured for other sensors.
- The system only monitors the soil; it does not water the plant by itself.

10. Future Scope

- Add a relay and a water pump to water the plant automatically when the soil is dry.
- Add LED or buzzer indication for local alerts without a phone.
- Store long-term data on a cloud database or SD card for daily and weekly trend analysis.
- Add a temperature and humidity sensor (for example DHT11) for complete plant-environment monitoring.
- Use battery and solar power with deep-sleep mode for field deployment.
- Allow thresholds to be changed from the dashboard or by Telegram commands.

11. Conclusion

The ESP32 Soil Moisture Monitoring System successfully measures soil moisture, converts it into a calibrated percentage, and presents it on a responsive live dashboard hosted by the board itself. At the same time it keeps the user informed through Telegram with dry alerts, recovery messages and on-demand status commands. Testing showed correct behaviour across dry, moist and wet conditions, and the hysteresis and cooldown logic prevented repeated alerts. The project demonstrates practical use of ADC sensing, Wi-Fi networking, an embedded web server and an internet messaging API, and it can be extended into a fully automatic irrigation system.

12. References

- Espressif Systems, ESP32 Series Datasheet and Arduino-ESP32 documentation.
- Brian Lough, UniversalTelegramBot library for Arduino.
- Benoit Blanchon, ArduinoJson library.
- Telegram Bot API documentation, core.telegram.org/bots/api.